Summary Section

Frasier Digital, LLC — a small disadvantaged business in Tomball, Texas (NAICS 541511) — proposes to finish, deploy, and maintain the CWMS data-authorization system for the Hydrologic Engineering Center with a named team of four individuals covering the five key personnel roles (one dual role per the Government's Q&A answer 9) plus two associate engineers, working from our own facility. Our approach starts from the repositories as they are: we land the open pull requests the previous effort left (cwms-data-api #1461, cwms-access-management #13), decide the three sources of truth the current baseline hard-codes (embargo, offices, constraints) through ADRs, expose the CWMS security schema's own time-series-group primitives through new CDA endpoints, and build the Technical Exhibit 1 administrator interface on top of them in six increments — all behind a feature flag so the office-plus-role model HEC runs today keeps working at every commit. In CWBI, the proxy, OPA, and cache run as sidecars in the existing CWMS-Data task, changed through Platform1 with fully specified requests. Load testing is a k6 harness in the repository that any developer runs locally and that scales to HEC's CloudStack environment, with the one-, two-, and three-instance topology as a configuration property. Maintenance follows the PWS clocks exactly, sized to HEC's review capacity.

This volume contains Factor 1 (Tab A Technical Approach, Tab B UI Improvements, Tab C Load Testing) and Factor 2 (Tab A Management Approach, Tab B Key Personnel, Tab C Transition Plan), followed by an annex of five resumes and five letters of commitment that do not count against the page limit. Frasier Digital, LLC is registered and active in SAM.gov (UEI PY8MJ4JPHJ45, CAGE 213L8; expires 27 May 2027), its Representations and Certifications are current, and no teaming partners are proposed. No price information appears in this volume.

Factor 1 — Response to Project Technical Approach

TAB A — Technical Approach

A.1 What we are inheriting, and what "done" means

HEC's decision to move authorization out of per-district Oracle Virtual Private Database policies and into the CWMS-Data-API (CDA) is the right one for a single cloud-hosted system, and the previous effort (W912HQ25P0049) produced a coherent design for it: a transparent proxy in front of CDA that resolves the caller's identity, asks Open Policy Agent (OPA) for a decision, caches it, and forwards the request to CDA with a single `x-cwms-auth-context` header that CDA's data layer turns into WHERE clauses on office, embargo, and sensitivity (PWS 1.2; ADR 0005, docs/source/decisions). We have read that design, the code that implements it, and the code that does not yet, and our technical approach starts from the specific state of the repositories rather than from the design on paper — because the two currently disagree.

Table A-1. The baseline as it stands in the repositories (verified August 2026).

Item	State	What finishing it requires	Task
cwms-data-api PR #1461 — `AuthorizationContextHelper`, `AuthorizationFilterHelper`, `TimeSeriesController`/`TimeSeriesDao` filtering, `UserDao` time-series-group privileges	Open since Nov 2025; 19 commits ahead of `develop`, **161 behind**; two of four schema-matrix CI legs and the OpenAPI static test failing; no approvals; unit tests only	Rebase onto `develop`; move header parsing into `ApiServlet` after the authenticator as a context attribute (the November review comment); apply the office filter in the DAO, not only the controller; return 403 rather than 400 on office denial; make the feature flag runtime-toggleable; split the podman/Keycloak infrastructure commits into their own PR; add `*IT` integration tests that prove allow **and** deny	2
ADR 0006 and `docs/source/access-management/**` on `develop`	Marked "Implemented" and documents the helper classes — which exist only on the PR #1461 branch	Reconcile documentation with merged reality in the same PR series; until then the docs describe behaviour CDA does not have	2
cwms-access-management PR #13 (login bounce: `token` vs `access_token`; memoized auth context; compose parity)	Open since Feb 2026	Merge with the compose file de-duplicated (see below); replace the short-lived access-token pattern the author flagged as "not quite correct" with refresh handling	2
`auth-contract` issues — 136 open across sub-epics 3a–3k	Titles only, no bodies; created as a plan, not as a backlog	Triage in the first two development meetings into: done-by-code, superseded-by-this-PWS, still-open; close with a comment, or attach acceptance criteria. Issue #1523 ("PWS delivery audit") is the right frame and we will complete it	1a, 5a

Item	State	What finishing it requires	Task
Compose vs. podman drift — `docker-compose.podman.yml` and `Dockerfile.podman` exist only on the PR #1461 branch; the UI repo is built against that layout (external `cwmsdb_net`, CDA on 7001, Keycloak on 8080) while CDA's `docker-compose.yml` runs Keycloak on 8081 behind Traefik, uses office `q0` not `s0`, starts Keycloak deliberately *after* CDA, and defaults the management flag off	Two incompatible local stacks	One compose file per repository with profiles for the variants that have a reason to exist (Q&A 36: "single items of truth"); podman-specific files removed unless a podman-specific reason survives review	2
Three hard-coded sources of truth — request constraints computed in the proxy's TypeScript rather than in Rego; office→division map hard-coded in `helpers/offices.rego`; embargo hours encoded in the time-series-group *name suffix* (`policy-dam_operator-r-72h`) after the `at_sec_ts_group_embargo` table was created and then dropped	None of the three is editable by a district administrator	Decide one home for each (Section A.2, Task 2) before building the UI that edits them	2
CDA principal cache, PR #1841 (`LruTtlMap`, open July–Aug 2026) and the proxy's Redis + in-memory decision cache	Two caches for overlapping data, neither with tests	One cache topology, defined in an ADR (Section A.2)	2, 3a
Rego tests	None (`opa test` coverage is zero); integration tests require five services and are documented as partially broken (Keycloak/DB username mismatch)	Policy unit tests with every persona and every deny path; integration-test fixture fixed	2, 5a

Two consequences follow. First, **the first thirty days are about landing what exists**, not adding to it; a proposal that started from "Task 2: build the UI" would be scheduling work on top of an unmerged foundation. Second, the effective authorization model today is the simple one the Government described in the Q&A — office plus role (`CWMS Users` may write, `TS ID Creator` may create identifiers, everyone else reads), enforced by `CdaAccessManager` and Oracle VPD — and **that model must keep working at every commit** while the policy-based model comes up behind a feature flag beside it.

A.2 Approach by task

Task 1a / 1b — Development meetings (PWS 2.1). Biweekly one-hour meetings with up to three of our staff relevant to the current work, and notes within one business day in the forms the PWS lists: typed minutes in Confluence, new or updated GitHub issues, milestones for intent, and skeleton pull requests with test names where a meeting produced a design decision. We treat the notes as the authoritative record for Task 5 approvals (Section A.2, Task 5) and as the running "PWS delivery audit". Our note template has four sections — decisions, work approved, blockers with owner, and next-meeting inputs — and every decision

that changes a design is also recorded as an ADR in `docs/source/decisions` so that other contributors can comment before acceptance (Q&A 46). Task 1b is priced as an additional 26-meeting quantity per Q&A 31.

Task 2 — Improve authorization and the web UI (PWS 2.1; TE1). Task B describes the UI work in detail. The sequence below is what makes the UI possible:

1. *Land PR #1461 behind the flag* (`cwms.dataapi.access.management.enabled`, made a Togglz feature so it can be toggled per environment without restart, consistent with how `USER_LISTS` is handled). The rebase includes the review items in Table A-1 and integration tests in the existing `DataApiTestIT` pattern using the `TestAccounts` persona users, with at least one test per persona that asserts a 403 and one that asserts filtered rows.
2. *Decide the three sources of truth and record them as ADRs.* Our recommendation, which we will bring to the first meeting with a written comparison: (a) **embargo** lives in the schema — reinstate `at_sec_ts_group_embargo(ts_group_code, embargo_hours)` from the previous effort's SQL, extend `av_sec_ts_group_mask`, and retire the name-suffix regex (`UserDao.EMBARGO_PATTERN`, issue #1541), which resolves the open reviewer question on PR #1461 about where an office's embargo rules come from; (b) **offices and regions** are loaded into OPA through its data API from `cwms_office`, replacing the hard-coded map, on a refresh interval; (c) **constraints** move from the proxy's TypeScript into Rego so that a policy is one artifact, testable with `opa test`, and the proxy becomes a thin decision client. Each is a small, reversible change and each removes a place where district-editable data is currently a code edit.
3. *Expose the security primitives CDA lacks.* The CWMS schema already has everything the TE1 Policy tab needs — `at_sec_ts_groups`, `at_sec_ts_group_masks` (a LIKE pattern over `Location.Parameter.ParameterType.Interval.Duration.Version`), `at_sec_allow`, and the `cwms_sec` package procedures `assign_ts_masks_to_ts_group` and `assign_ts_group_user_group` — but CDA exposes none of it; the existing `/timeseries/group` endpoints are the data-categorization groups, not the security groups. We add `/auth/ts-groups/{id}`, `/masks`, and `/user-groups` controllers under `api/auth/`, registered in `ApiServlet.addUserManagementHandlers()` with the `CWMS User Admins` role, backed by jOOQ bindings to `cwms_sec` in the same style as the existing `CWMS_TS_PACKAGE` calls. District-scoped role administration uses the `POST/DELETE /user/{user-name}/roles/{office-id}` endpoints CDA already has and the management UI does not yet call.
4. *Build the UI increments in TE1 order* (Tab B), each as a work unit with tests, and the documentation and training materials as the last increment of the 180-day period.

The minimum successful product the PWS defines — an interface to manage access control, the access-control method working for time-series data, and documentation — is reached at the end of step 3 plus the first two UI increments; the remainder of Task 2 is breadth.

Task 3a / 3b — Stand up the authorization system in CWBI-Dev, then CWBI-Test (PWS 2.1; TE2). CWMS-Data already operates inside the CWBI boundary as an ECS/Fargate module with an internal ALB, RDS Oracle, and Keycloak as the identity provider, defined in Python CDK (Q&A 37; CWBI standard-app pattern). We cannot create services; every infrastructure change is a Platform1 Jira issue to the CWBI infrastructure contractor, with the possibility of pull requests to the dev CDK (Q&A 11). Our approach is therefore to arrive at the first Platform1 ticket with the change fully specified:

- *Container placement.* The proxy, OPA, and cache run as additional containers in the CWMS-Data task definition — sidecars to CDA, sharing its task network — rather than as separate services. That keeps the `x-cwms-auth-context` header on a localhost hop, which matters because the header is unsigned (Section A.4). We will document the alternative (separate services behind path-based ALB listeners) in the same ADR with the cost and maintainability comparison Q&A 35 asks for, and recommend the sidecar layout unless CWBI's task-size limits force the split.

- *Cache.* CWMS is creating a Redis/Valkey cache in the environment for other needs (Q&A 35); we treat it as given, key the decision and user-context entries as the proxy does today, and retire the proxy's in-memory map and the CDA-side `LruTtlMap` so that there is one cache with one TTL policy and one invalidation path (user role change → evict `user:context:{username}`).
- *Images and health.* Hardened images per CWBI (Chainguard where an image exists; otherwise distroless or an image with a documented update path — Q&A 10), a `/health` endpoint on every container for CWBI's uptime monitoring (the proxy and OPA have them; CDA does not — we add `GET /cwms-data/health` so the target-group check stops depending on `/offices/HEC`), and the CWBI logo and consent banner on the management UI (TE2 Infrastructure).
- *STIG posture.* We self-assess against the Application Security and Development STIG and the Application Server STIG before each Dev deployment and keep the checklist in the repository; CWBI scans, we respond within the timelines in TE2, and the ISSO/ISSM asks for any ATO artifact through email or the Jira issue (Q&A 10, 37). No eMASS interaction, no on-site SCA, no travel (Q&A 12, 51).
- *Identity.* Keycloak with Login.gov or EAMS-A as CWBI directs (TE2 Infrastructure); CDA's `OpenIdConnectIdentitityProvider` already consumes Keycloak JWTs, and the proxy's decode-without-verify shortcut is replaced with JWKS verification against the same well-known endpoint.
- *Definition of done.* Task 3a: a CWBI-Dev deployment where a persona user's request through the public FQDN is decided by OPA, cached, and filtered by CDA, with the flag on; the same request with the flag off behaves exactly as today. Task 3b repeats the promotion into CWBI-Test with the Test-environment configuration in the CDK environment YAML.

Task 4 — Load testing (PWS 2.1; TE3). Tab C.

Task 5a / 5b — Maintenance (PWS 2.1; TE4). Maintenance is a workflow with clocks, and we run it as one. A request arrives as a GitHub issue; we start within five days and submit a first work unit within two days of starting (PWS Task 5a). Classification takes up to one hour and ends with a status comment (bug — minor/routine/major; misuse; functionality gap) and an estimate; work over two hours waits for the approval label from the Technical POC, Technical Supervisor, or designee; a draft pull request is pushed after four hours of work regardless of completeness; every fix carries the description of the testing performed, and the monthly invoice references the issue with the six log fields the PWS requires (requestor, date, work-unit notes, issue status, fix description with tests, hours). Because the Government pays for delivered and accepted work units rather than in equal monthly amounts (Q&A 33), our maintenance staffing is fractional and elastic — a named associate engineer carries the steady-state load with a senior engineer behind him for major fixes — and weekly volume is tuned in the biweekly meeting against HEC's review capacity (PWS Rate of Work; Q&A 47). Scope is the repositories the Government named: cwms-data-api, cwms-access-management, and cwms-database primarily; cwms-python, cwms-cli, and cwms-data-api-client if a fix requires it; and upstream libraries through their own bug process with HEC's support (Q&A 46).

Work-unit discipline across every task. Pull requests are sized to HEC's review ladder — up to three files or 200 lines for a three-day review; up to ten files or 500 lines for five; anything larger for ten — and we prefer the first rung. Tests, or proof that existing tests cover the change, accompany every code change; new tools and helpers for loading test data follow the existing `defaultload.txt`/`timeseries.csv` fixture pattern; and the test-location preference order in the PWS (JUnit 5 unit tests, the JUnit 5 integration suite, anything repeatable locally, then anything repeatable on the build server) is the order we use.

A.3 Working with HEC, the CWBI-PMO, and other contractors

Three interfaces, three cadences. With HEC's Technical POC: pull-request review at the PWS ladder, the biweekly meeting, and a shared channel for same-day questions; we submit small units and we do not queue more than HEC can review in a week. With the CWBI-PMO: Platform1 Jira issues for every infrastructure change, written to their template with the standard-app diagram attached; response to scan results and

STIG-state requests inside the TE2 timelines; ISSO/ISSM artifact requests answered from the repository's security folder. With other contractors and USACE staff working in the same files: the Government's rule is that the first approved merge wins and the other party resolves the conflict (Q&A 46) — we rebase daily on `develop` to keep our side of that small, and anything beyond a source conflict goes to the next meeting or, if it cannot wait, to the Technical POC and COR the same day (PWS Coordination). Design-bearing disagreements become ADRs that all parties can comment on before acceptance.

Onboarding path (Q&A 11, 13, 19). Week 1: DD 7798 and DD 2875 for every named person, Public Trust initiation, the Platform1 access request, and delivery of Frasier Digital's fixed egress IP address to CWBI for bastion-host access (a dedicated static-IP egress provisioned for this contract). After kickoff: GitHub Enterprise repository access, expected within a day or two. Platform1: a week or more; bastion access: up to a month. None of that gates the first month of work: the CDA, access-management, and database repositories are public, the docker-compose stacks run on a laptop with the `gvenzl/oracle-free` image (Q&A 18), read access to the CDK is available immediately (Q&A 19), and the PR #1461 rebase, the ADRs, and the first UI increments are all local work. Our transition-in schedule (Factor 2, Tab C) shows the two tracks side by side.

A.4 The knowledge areas the Government will evaluate (instr. 3.2.1.1.1)

Full life cycle and agile practice. Two-week iterations aligned to the meeting cadence; a definition of done that equals an accepted work unit; every increment behind a feature flag until HEC turns it on; GitHub Actions as the build server — CDA's `build.yml` already runs the four-schema matrix, the integration tests, the OpenAPI static test, and the nightly ECR push through OIDC, so our CI additions (Rego tests, the k6 harness, the proxy's vitest suite) are steps in that workflow, not a parallel system. The TeamCity migration the PWS mentions is complete; only naming vestiges remain and we will remove them as we touch those files.

Database administration. CWMS is Oracle today and will migrate data to storage matched to its use (PWS 1.2); the authorization work must not deepen the Oracle dependency. Our Senior Oracle DBA owns the schema changes the design needs (the embargo table, the mask view extension, any `cwms_sec` procedure change), reviews the VPD interaction so that CDA-level filtering never contradicts a database-level policy, and keeps the integration-test images (`schema_installer:latest-dev`, `database-ready-ora-23.5`) current with the schema PRs in cwms-database. RDS-specific behaviour in GovCloud — parameter groups, the block-storage throttling the Government has observed under bursty writes (Q&A 38) — is part of the Tab C baseline.

REST API development. CDA is Javalin 4 with an `IdentityProvider` SPI and an `AccessManager`; the correct place for header parsing is the servlet's `before` chain, once, into a context attribute (the review comment on PR #1461), and the correct place for row filtering is the DAO. New endpoints ship with OpenAPI annotations so the static analysis test passes, and the API remains consumable by external clients (TE2 Data Exchange) — the cwms-python and JavaScript clients are among the repositories in scope.

Access-control mechanisms. Deny by default; OPA and the policy concept remain the decision point because further modules will integrate with it (Q&A 44); the policy model is district → user group → time-series group → mask → privilege → embargo, which is the CWMS security schema's own model, so the UI edits real rows rather than a parallel store. Authentication stays with Keycloak and CDA's existing providers (OIDC, API key, CAC where enabled). Tests prove both outcomes for every persona.

Configuration management. GitHub Enterprise for everything the Government funds; one compose file per repository; feature flags through Togglz; ADRs for design changes; container base-image updates as a recurring maintenance work unit so that "reasonably easy to update libraries and images" (Q&A 10) is demonstrated, not asserted.

Cybersecurity, and communicating it succinctly. OWASP ZAP runs in CI against the compose stack on every pull request that touches request handling; the OWASP Top Ten review is part of our pull-request

template. Four specific weaknesses in the current baseline are closed in Task 2, and we state them here because the evaluators asked for the ability to communicate security needs succinctly: (1) CDA does not verify that `x-cwms-auth-context` came from the proxy, so anything that reaches CDA directly can forge or omit it — we sign the header with an HMAC shared between proxy and CDA (sidecar placement makes the key exchange local), and treat a missing header as deny when the flag is on; (2) the proxy honours an `x-test-user` impersonation header that is not gated by environment — removed outside test profiles; (3) the proxy decodes JWTs without verifying them — replaced by JWKS verification; (4) `BYPASS_AUTH=true` fails open — replaced by fail-closed with an explicit break-glass procedure. Audit records carry event type, time, source, outcome, and subject (TE2 Infrastructure), and are reviewed weekly during Task 5.

CWBI processes and requirements. Platform1 change requests; hardened images; health endpoints; the standard-app layout (Fargate service, internal ALB, RDS, Keycloak, WAF in front); secrets in Secrets Manager with ARNs in SSM; a minimal PMP with the standard-app diagram for our module (Q&A 48, 50); API registration in the CW Data Catalog (TE2 Metadata); no CUI or CDI in the work (Q&A 14).

A.5 Proposed level of effort per task (instr. 2.1.1)

Hours below reconcile exactly with Volume III and with the staffing plan in Factor 2, Tab A. No price information is included in this volume.

Table A-2. Level of effort by task and role (hours).

Task	PM	Sr Oracle DBA	Sys Eng / Architect	Sr Forms (React) Dev	Sr Java Dev	Associate Engineers	Total
1a Development meetings (26)	78	8	12	12	12	8	**130**
2 Authorization + UI	50	30	—	220	150	100	**550**
3a CWBI-Dev stand-up	40	—	120	—	40	—	**200**
5a Maintenance (≤803)	70	60	—	100	150	400	**780**
Base total	238	98	132	332	352	508	**1,660**

Task	PM	Sr Oracle DBA	Sys Eng / Architect	Sr Forms (React) Dev	Sr Java Dev	Associate Engineers	Total
1b Development meetings (option)	78	8	12	12	12	8	**130**
3b CWBI-Test stand-up (option)	20	—	80	—	20	—	**120**
4 Load testing (option)	20	20	60	—	40	40	**180**
5b Maintenance (option, ≤704)	60	50	—	90	130	350	**680**
All-options total	416	176	284	434	554	906	**2,770**

Task 5a and 5b hours are ceilings for planning; the Government's answer that payment follows delivered and accepted work units (Q&A 33) is reflected in the elastic staffing described in Task 5 above.

TAB B — UI Improvements

B.1 The current access-management application

The `cwms-access-management` repository is an Nx monorepo (pnpm, Node 24, TypeScript 5) containing a Fastify authorization proxy, a Fastify management API, a Commander/Ink CLI, and a React 18 management UI built with Vite, Tailwind, TanStack Query, react-router, and zustand. The UI has five pages — Home, Login, Users, Roles, Policies — and it is, in its entirety, **read-only**. The Users page is a table of username, email, full name, and an Active badge with client-side search; the `User` type in the UI's API service omits the `offices` and `roles` fields that the management API actually returns, so district scope is invisible. The Roles page lists role names and IDs; every description renders as "–" because the flat `GET /roles` in CDA returns only distinct group identifiers with no office. The Policies page lists Rego files read from OPA's `/v1/policies` endpoint and renders them with a syntax highlighter. The management API answers `POST` and `DELETE` on users and roles with HTTP 501. A Keycloak admin service exists in the code and is not wired to anything. In short: the previous effort delivered the frame and the read paths; the write paths, the district scoping, and the policy editing that TE1 describes do not exist yet.

Three further shortcomings shape the design, all traced to specific files:

1. **District scope has no representation in the UI or the management API.** CDA's own user endpoints (`GET /users?office=…`, `POST/DELETE /user/{user-name}/roles/{office-id}`) are office-scoped and already enforce the `CWMS User Admins` role; the management API calls `GET /users?include-roles=true&page-size=1000` once and flattens roles across offices. Every TE1 requirement that says "for their district" depends on restoring that scope end to end.
2. **Policy data is not data.** Offices and regions are a hard-coded map in `policies/helpers/offices.rego` (with a comment promising to load them "in a future sprint"); embargo hours are parsed from a naming convention on time-series-group identifiers; per-persona constraints are computed in the proxy's TypeScript. A Policy tab that lets a district administrator "create policy and perform at least basic validation" cannot be built against files that are mounted into the OPA container and require a container restart to change.
3. **There is no CDA endpoint for the security time-series groups.** The schema has them — `at_sec_ts_groups`, `at_sec_ts_group_masks`, `at_sec_allow`, and the `cwms_sec` procedures that manage them — and the `/timeseries/group` endpoints look like the right thing but are the data-categorization groups in `at_ts_group`. "Assign a time series to a policy" therefore begins in Java, not in React.

The shortcomings are ordinary for a first increment, and they define the order of work.

B.2 Technical Exhibit 1, requirement by requirement

Table B-1. Gap analysis against TE1. *Exists* = works today; *Partial* = data path exists, UI does not; *Missing* = requires new CDA endpoint(s) or a source-of-truth decision.

TE1 requirement	State	Where the change lands	Increment
Users tab			
View all users with roles/permissions for their district	Partial	UI `UsersPage`, API `getUsers(office)` → CDA `GET /users?office=&include-roles=true`	U1
Assign roles to users for a specific district	Partial	UI assign control → mgmt API (new write route) → CDA `POST /user/{u}/roles/{office}` (exists)	U1
Show new users that do not yet have privileges	Missing	CDA `GET /users?unassigned=true&office=` (new param), UI "Unassigned" filter	U2 (§B.4)

TE1 requirement	State	Where the change lands	Increment
Organize users by assigned roles	Partial	UI grouping on the roles field the API already returns	U1
Cannot assign roles for districts they lack permission on	Partial	CDA enforces via `is_user_admin(office)`; UI limits the office selector to the admin's offices from `/user/profile`	U1
Roles tab			
Create roles specific to a district's data	Missing	CDA endpoint wrapping `cwms_sec.create_user_group` / `delete_user_group` (office-scoped); UI create form	R1
Assign policies to roles	Missing	Policy = time-series-group grant (`assign_ts_group_user_group`) — see Policy tab; UI role detail page lists grants	R2
Sort/filter roles by assigned policies	Missing	Follows R2 data	R2
View/edit/filter roles for a district	Partial	`GET /roles` gains office scope (from `av_sec_user_groups`); UI filter	R1
Cannot edit roles for districts they lack permission on	Partial	CDA role check; UI scope	R1
Policy tab			
View policies associated with district data	Partial	Today: Rego viewer. After P1: list of security time-series groups with masks, grants, embargo for the district	P1
Assign a time series to a policy	Missing	CDA `/auth/ts-groups/{id}/masks` (new) → `assign_ts_masks_to_ts_group`; UI selector	P1 (§B.3)
Use time-series groups (or manage within the authorization system); restrictive or allowed	Missing	Security groups *are* the mechanism; allowed vs restrictive expressed as read/write/none grant per user group; UI toggle	P1
Assign by time-series parameters (LOC/parameter/version)	Missing	Mask pattern over the six-part identifier; UI mask builder with catalog preview	P1 (§B.3)
Assign embargo to a time series	Missing	Embargo column/table decision (Tab A, ADR); CDA field; UI hours picker	P1 (§B.3)
Create policy and perform basic validation	Missing	CDA validation (mask syntax, office ownership, group-name uniqueness) + UI inline validation	P1
Cross-cutting			
Bulk manipulation of permissions (PWS Task 2)	Missing	Multi-select on Users and Policy tabs; batched CDA calls with per-row result	U3
Section 508 (Q&A 49)	Partial	Keyboard paths, labels, focus order, error messaging; Storybook stories per component	every increment

Increment order across the 180-day Task 2 period: **U1 → P1 → U2 → R1 → R2 → U3**, with documentation and training materials in the final increment. The PWS minimum successful product — an interface to manage access control and the access-control method working for time-series data — is reached at the end of P1.

B.3 One requirement in detail: assign a time series to a policy, scoped by location, parameter, and version, with an embargo

This is the requirement that touches every layer, so it is the one we take all the way down.

What a district administrator is doing. SWT's water manager wants raw stage data at a set of gauges to be readable by the district's cooperators only after 72 hours, while the district's own operators see it immediately. In CWMS terms: a security time-series group scoped to `*.Stage.Inst.*.0.Raw*` at those locations, granted *read* to the `external_cooperator` user group with a 72-hour embargo, and *read/write* to `dam_operator` with none.

Data model. We use the schema's own model rather than inventing a parallel one, because the database, VPD, and CDA's `UserDao` already understand it:

- `at_sec_ts_groups(db_office_code, ts_group_code, ts_group_id, description)` — the policy, owned by an office.
- `at_sec_ts_group_masks(db_office_code, ts_group_code, ts_group_mask)` — one or more LIKE patterns over the six-part time-series identifier `Location.Parameter.ParameterType.Interval.Duration.Version`; the location/parameter/version scoping TE1 asks for is a mask with wildcards in the other positions, normalized by `cwms_util` (`*` and `?` accepted).
- `at_sec_allow(db_office_code, ts_group_code, user_group_code, privilege_bit)` — the grant: read (2), write (4), or none.
- `at_sec_ts_group_embargo(ts_group_code, embargo_hours)` — reinstated from the previous effort's `001-ts-group-embargo.sql`, replacing the name-suffix convention (Tab A, ADR). `av_sec_ts_group_mask` and `av_sec_ts_privileges` are extended to carry the hours so that one view answers "what may this user see, and from when".

Policy evaluation. The proxy already sends OPA `{user:{offices, roles, ts_privileges[]}, resource, action, context}`; `ts_privileges` gains `embargo_hours` from the view instead of from a regex, and `helpers/time_rules.rego` — which already contains `get_ts_group_embargo_hours` — reads it. The decision returned to CDA carries `ts_group_embargo` in the constraints block; `AuthorizationFilterHelper.getTsGroupEmbargoFilter` (present in PR #1461, never called) is wired into `TimeSeriesDaoImpl.getRequestedTimeSeries` beside the office filter, producing `date_time < now - embargo_hours` for matching identifiers. Embargoed rows are absent, not zeroed; the response's paging cursor is computed after the filter so that clients see a consistent window. A Rego unit test file per persona asserts the allow, the deny, and the embargo boundary at ±1 second.

CDA endpoints (new, `api/auth/`, role `CWMS User Admins`).

Method and path	Backed by	Purpose
`GET /auth/ts-groups?office=`	`av_sec_ts_group_mask`, `av_sec_ts_privileges`	List policies for a district with masks, grants, embargo
`POST /auth/ts-groups`	`cwms_sec.create_ts_group`	Create a policy (validates name uniqueness within office)
`PATCH /auth/ts-groups/{id}`	`change_ts_group_desc`, embargo table	Edit description or embargo hours
`PUT /auth/ts-groups/{id}/masks`	`assign_ts_masks_to_ts_group` (add/remove lists)	Set the location/parameter/version patterns
`PUT /auth/ts-groups/{id}/user-groups/{ug}`	`assign_ts_group_user_group` with `read`, `write`, or `none`	Grant or revoke
`GET /auth/ts-groups/{id}/preview?office=`	`GET /catalog/TIMESERIES` with `like=`	Show which identifiers a mask currently matches

Each is a jOOQ call into `cwms_sec` in the style of the existing `CWMS_TS_PACKAGE` bindings, with DTOs under `data/dto/auth/` (reusing `TsGroupPrivilege` from PR #1461), OpenAPI annotations so the static-analysis test passes, and an `*IT` test class in the `DataApiTestIT` pattern that creates a group as a `CWMS User`

`Admins` user, assigns masks and a grant, then reads time series as an `external_cooperator` and asserts that rows inside the embargo window are absent and rows outside it are present.

Management API. A `routes/ts-groups.ts` that forwards the six calls with zod validation (`middleware/validation.ts`), and the office-scoped `getUsers`/`getRoles` changes from increment U1. The management API stops flattening roles across offices.

React UI — the Policy tab flow.

1. *Office selector* limited to the offices where `/user/profile` says the administrator holds `CWMS User Admins`. Everything on the page is scoped to it.
2. *Policy list* (replaces the Rego viewer as the default view; the viewer moves to an "Advanced" panel): name, description, mask count, grants, embargo, last modified.
3. *Create / edit policy* form: name, description, embargo hours (0 = none) — inline validation from the CDA response.
4. *Mask builder*: three fields — Location (with type-ahead from `/locations?office=`), Parameter (from `/parameters`), Version (free text) — and a generated pattern shown as text (`*.Stage.Inst.*.0.Raw*`) that an advanced user can edit directly. A **preview** panel calls the preview endpoint and lists the identifiers the pattern currently matches, with a count, so that an administrator sees the effect before saving. Multiple masks per policy.
5. *Grants*: a table of the district's user groups with a three-state control (none / read / read+write) per row; changes are batched into one `PUT` per changed row with per-row success or error shown in place.
6. *Bulk assignment* (increment U3): select several policies or several time series from the catalog and apply a grant or an embargo in one action, with a result table.

Components are built with Storybook stories and React Testing Library tests; keyboard operation, visible focus, labelled controls, and error text tied to fields by `aria-describedby` are part of each component's definition of done (Q&A 49 — HEC requires 508 conformance and uses Storybook; full pass on every rule is not required, but the interactive path must work without a mouse).

What "working" looks like at the end of P1. A district administrator with no command-line access creates the SWT policy above through the UI; a cooperator's `GET /cwms-data/timeseries?name=…&office=SWT` returns data older than 72 hours and nothing newer; an operator's identical request returns everything; with the feature flag off, both requests behave exactly as before the change. That scenario is the acceptance test, and it is also the first walkthrough in the training materials.

B.4 A second requirement, in brief: show new users who have no privileges yet

With `cwms.dataapi.access.openid.create_users=true` — set in both compose files — a person's first Keycloak login creates an `at_sec_cwms_users` row with no `at_sec_users` membership. Those are the users TE1 wants surfaced, and today they are invisible: `UserDao.getAll` deliberately restricts its result to users who already have a group membership. The change is one query parameter and one join: `GET /users?office=&unassigned=true` implemented as `at_sec_cwms_users` left-joined to `av_sec_users` on office, returning rows with no membership (or, optionally, no persona-group membership), with a matching `UsersControllerTestIT`. The management API forwards the parameter; the Users page gains an **Unassigned** filter beside the office selector and an *assign role* control on each row that calls the existing `POST /user/{user-name}/roles/{office-id}`. Registration can happen at any time, so the list is live, not a snapshot.

B.5 How the UI work progresses through the contract

Increment U1 (district scope restored, role assignment working) ships in the first month alongside the PR #1461 rebase, because it exercises only endpoints CDA already has. P1 follows once the embargo ADR is accepted, since its schema change is the one decision that cannot be reversed cheaply. U2 and R1 are small

and fill review gaps. R2 and U3 complete the exhibit. Documentation — an administrator guide organized by the three tabs, and a short training deck with the SWT walkthrough — is the final Task 2 work unit, delivered in the repository's `docs/source/access-management/management/` tree where the previous effort left placeholders.

TAB C — Load Testing

C.1 What the Government asked for, restated as design constraints

TE3 and the Q&A answers give six constraints, and each one narrows the design:

1. **Any developer must be able to run the tests locally**, even where local numbers do not reflect production (TE3; Q&A 38) — so the harness cannot depend on cloud infrastructure to exist at all.
2. **The same tool must scale to an environment of "more verisimilitude"** as an option, not a rewrite (Q&A 38) — HEC's Apache CloudStack environment at no cost, or another if demonstrably better (Q&A 39).
3. **Open-source, unrestricted licensing** (Q&A 38).
4. **The target of interest is CDA response time**, regardless of the calling application, so that caching strategies can be analysed (Q&A 38); the stated intent is to learn whether RDS needs vertical scaling or read replicas and whether CDA benefits from horizontal scaling (TE3).
5. **One, two, and three CDA instances** (round-robin) must each be testable, but as a configuration property rather than a mandatory triple run (Q&A 40); CPU/memory fixed at 1 vCPU / 2 GB per CDA instance (TE3).
6. **This is baseline characterization, not performance targets** (TE3; Q&A 40): the five load rows are controlled attempts, coverage of read/write and authenticated/unauthenticated combinations should be "as many as practical" starting from a reasonable sample, and the minimum facilities are authorization, time-series storage and retrieval with catalogs, and location storage and retrieval with catalogs.

C.2 Tool: k6, and why

TE3 names Apache JMeter as an example; the previous effort's design document listed JMeter but its repository actually contains **k6** scripts (`tools/benchmark/scenarios.js`, `quick-benchmark.js`, `run-benchmark.sh`) with Keycloak password-grant tokens for the persona test users and thresholds on p95/p99 latency. We propose to keep k6 and build the harness on it, for four reasons that map to the constraints above:

- **Local by default.** k6 is a single static binary; a scenario is a JavaScript file; a run is one command against `http://localhost:8081/cwms-data`. Nothing about it assumes a cluster.
- **Scales without changing the artifact.** The same script runs distributed (k6 in Docker across CloudStack VMs, or the open-source `k6-operator` on Kubernetes) — the "option within the same tool" the Government described.
- **Open source, Apache 2.0**, with no GUI-bound test definitions to version-control.
- **It is already there.** Continuity with the existing scripts, tokens, and report format (`docs/benchmarks/`), which HEC has seen.

JMeter remains a legitimate alternative and we would switch if HEC prefers it: the scenario matrix in C.4 is tool-neutral and the harness layout keeps the runner behind one script so that a JMeter `.jmx` set could replace the k6 files without touching the rest. What the previous scripts lack — and what Task 4 adds — is the multi-instance topology, the write paths, the data-seeding step, and the result analysis.

C.3 Harness layout

Everything lives in the cwms-data-api repository under `load-tests/` so that the code under test and the tests that load it are versioned together and the harness is maintained under Task 5 like any other code.

```
load-tests/
  README.md                how to run locally, against CloudStack, against CWBI-Dev (read-only)
  config/
    local.json             target base URL, instance count, auth mode, data set, durations
    cloudstack.json
```

```
    cwbi-dev.json           read-only scenarios only; never write to a shared environment
  scenarios/
    auth.js                 authorization mechanisms: unauthenticated, API key, OIDC bearer,
via-proxy vs direct
    timeseries.js           GET /timeseries (several window lengths), POST /timeseries, catalog
TIMESERIES
    locations.js            GET /locations, GET /locations/{id}, catalog LOCATIONS, POST
/locations
    ratings.js              POST /ratings/rate-values and /rate-ts (the "rate" endpoint)
    mix.js                  read-heavy composite (the Government's stated assumption)
  lib/
    tokens.js               Keycloak password grant for persona users; API-key header helper
    data.js                 identifier lists generated from the seeded data set
    windows.js              begin/end generators for 1 h, 1 d, 7 d, 30 d, 365 d windows
  seed/
    seed.py                 loads the load_data/ CSV and parquet fixtures (LRL, MVP) into the
target DB
  topology/
    docker-compose.load.yml overlay: `--scale data-api=N` behind Traefik round-robin;
OPA/proxy/cache profiles
  run.sh                    one entry point: run.sh <config> <scenario> [--instances N] [--auth
mode] [--profile proxy]
  results/                  JSON summaries per run; analysis notebook reads them
  analysis/
    baseline.ipynb          p50/p95/p99, throughput, error rate by scenario × instances × auth ×
window
```

Targeting a specific system (TE3: "reasonably easy for a developer to target a specific system") is the `config/*.json` file plus `--instances`; nothing else changes between a laptop and CloudStack.

Instance topology. CDA's compose file already carries the Traefik load-balancer label (`traefik.http.services.data-api.loadbalancer.server.port=7000`); the load overlay adds `--scale data-api=N` and a health-checked round-robin so that one, two, or three instances are a number in the config, exactly as Q&A 40 asked. Each instance is pinned at 1 vCPU / 2 GB (TE3) through compose resource limits, and the same pins carry to CloudStack VMs. The authorization profile adds the proxy, OPA, and cache containers in front so that "via proxy" and "direct to CDA" can be compared on identical runs — the difference is the authorization overhead the design is meant to keep under a few milliseconds.

Data. The repository's `load_data/` notebooks and the LRL/MVP location and melted-time-series fixtures give a realistic seed; `seed.py` loads them once per environment and generates the identifier lists the scenarios draw from, so every developer runs against the same data shape. Write scenarios use a dedicated office and identifier prefix and clean up after themselves; the `cwbi-dev.json` config disables writes outright.

C.4 Scenario matrix

Table C-1. Load rows from TE3 × topology × mode. Every cell is a config combination, not a separate script; the run script iterates the ones selected.

TE3 load row	Users / rate / parallelism	Instances	Auth modes	Read/write mix
L1	1 user, 50 req/s, parallelism 10	1, 2, 3	unauth, API key, OIDC, via-proxy	read; write
L2	100 users, 50 req/s each, parallelism 10	1, 2, 3	unauth, OIDC, via-proxy	read-heavy mix (90/10)
L3	1,000 users, 100 req/s aggregate	1, 2, 3	OIDC, via-proxy	read-heavy mix
L4	10,000 req/s (controlled attempt)	1, 2, 3	unauth, via-proxy	read
L5	100,000 req/s (controlled attempt)	3	unauth	read (catalog + timeseries)

Per Q&A 40, L1–L3 are interpreted as a combination of parallel and sequential virtual users; L4 and L5 are attempts whose purpose is to find where the stack saturates (CDA CPU, connection pool, RDS I/O, or the load generator itself — the harness records which). Each scenario runs the facilities TE3 lists as minimum: authorization (token acquisition and a decision-bearing request), time-series `GET` across five window lengths and one `POST`, catalog of time series, location `GET`/`POST` and catalog of locations, and the rating endpoints. Coverage of the remaining endpoints is added as the sample grows; TE3 accepts a single `POST` and `GET` per mechanism for this baseline and does not require 100 % (Q&A 40).

Runs per cell are short (60–120 s steady state after a 30 s ramp) so that a full local sweep of the read cells fits in an evening on a laptop, and a CloudStack sweep including L4/L5 fits in a day.

C.5 What the baseline answers, and what we deliver

The Government named three questions and two observations, and the analysis notebook is organized around them:

- **RDS vertical vs. read replica.** Compare RDS-side latency and I/O across L1–L3 with one CDA instance (isolating the database) against two and three instances (adding API concurrency); a database-bound curve flattens as instances rise.
- **CDA horizontal scaling.** Throughput and p95 per added instance at fixed load; the point where a third instance stops helping is the database or the proxy, and the via-proxy/direct comparison says which.
- **Caching strategy.** Via-proxy runs with a cold and a warm decision cache show the authorization overhead and the cache hit rate; the previous effort's single-instance measurement (about 2–3 ms proxy overhead, 99.7 % Redis hit rate on a developer machine) becomes a multi-instance figure on shared infrastructure.
- **"Write amplification" and bot downloads (Q&A 38).** Two dedicated scenarios: hourly-style write bursts (GOES-shaped, small and frequent) followed by computations, to reproduce the block-storage throttling HEC has observed; and a small number of clients pulling large time-series windows continuously, to characterize what a download bot costs the rest of the system. Both feed the caching discussion — a read-through cache in front of catalog and long-window reads is the obvious candidate and the numbers will say whether it is worth it.

Deliverables (Task 4): the harness and overlay in the repository with a README that a new developer can follow in under an hour; seeded data and identifier lists; a results folder with the JSON summaries of the baseline sweep on the local stack and on CloudStack; the analysis notebook and a short baseline report (tables of p50/p95/p99, throughput, error rate, and saturation point per cell, with the three questions answered as far as the data allows and the next experiments suggested); and a GitHub Actions job that runs a smoke subset of the read scenarios on every pull request to `develop` so that the harness does not rot between uses. No performance targets are asserted — TE3 is explicit that there are none — but the harness is built so that HEC can set them later and let CI enforce them.

Factor 2 — Management Approach, Key Personnel, and Transition Plan

TAB A — Management Approach

A.1 Organization and lines of authority

Frasier Digital, LLC is a principal-led small disadvantaged business. The Project Manager is the company's founder and managing member; there is no account layer between the Contracting Officer's Representative and the person with authority over staffing, priorities, and corrective action. The organization for this contract is shown in Figure A-1.

Figure A-1. Organization chart and lines of communication.

<table>
<tr><th colspan="2">Government</th><th>Contracting</th></tr>
<tr><td colspan="2">USACE HEC — Technical POC / Technical Supervisor
CWBI-PMO — Platform1, ISSO/ISSM
code review · priorities · approvals · scans</td><td>CEHEC-CT — Contracting Officer / Contract Specialist
contract administration · invoices</td></tr>
<tr><td colspan="3">Joseph Frasier — Project Manager (KEY)
single point of contact · owns every deliverable · chairs biweekly meetings · approves every submission</td></tr>
<tr><td>Scott Carpenter
System Engineer / Architect (KEY)
CWBI, CDK, sidecars (proxy, OPA, cache), images, STIG</td><td>Randy Chong
Senior Java Developer (KEY)
CDA, PR #1461, <code>auth/</code> endpoints, integration tests</td><td>Zachary Antosko
Senior Forms (React/TypeScript) Developer + Senior Oracle DBA (KEY, dual role — Q&A 9)
management UI, management API, TE1 increments · schema, VPD interaction, test images</td></tr>
<tr><td colspan="3">Delivery pod (non-key) — reports to the PM; day-to-day direction from the senior owner of each stream
Andrew Frasier, associate engineer — Task 2 UI components · Seth Chesky, associate engineer — Task 5a maintenance and tests</td></tr>
</table>

Authority runs one way: every team member reports to the Project Manager, who is the only person who commits the company to the Government. Communication runs on one shared channel visible to the whole team, so that a question asked of any engineer is seen by the PM the same day. Technical direction from HEC arrives through pull-request review, GitHub issues, and the biweekly meeting; we do not create side channels that the Government cannot see.

A.2 Staffing plan and simultaneity

The team is named, confirmed, and sized against the level of effort in Factor 1, Table A-2, which Volume III prices. Table A-3 shows how the base-period tasks run concurrently across the 360-day period of performance without any task starving another: Task 2 and Task 3a run in parallel through month six on different people (the UI and CDA developers on Task 2; the architect on Task 3a), Task 5a runs at a steady fractional rate from the first approved issue, and Task 1a is a fixed cadence.

Table A-3. Base-period staffing by task and quarter (average hours per week; reconciles with Factor 1, Table A-2 base hours).

Person (role)	Q1 (days 1–90)	Q2 (91–180)	Q3 (181–270)	Q4 (271–360)	Tasks
Joseph Frasier (PM)	6	5	4	4	1a, 2, 3a, 5a
Scott Carpenter (Architect)	5	4	1	—	1a, 3a
Randy Chong (Sr Java)	8	7	6	6	1a, 2, 5a
Zachary Antosko (Sr Forms Dev + Sr Oracle DBA)	12	11	6	4	1a, 2, 5a
Andrew Frasier (associate)	4	4	—	—	2

Person (role)	Q1 (days 1–90)	Q2 (91–180)	Q3 (181–270)	Q4 (271–360)	Tasks
Seth Chesky (associate)	6	8	10	8	5a
Team total	**41**	**39**	**27**	**22**	

Task 5a's weekly volume is held under the 20-hour planning figure the PWS suggests and is tuned in each biweekly meeting to HEC's review capacity (PWS Rate of Work; Q&A 47). Option tasks add hours without adding people: Task 3b reuses the architect once 3a is stable; Task 4 uses the architect and the Java developer with the associate engineers running sweeps; Task 5b continues the 5a pattern.

Why this staffing does not degrade under concurrency. Three properties, each structural rather than aspirational: (1) the two build streams — UI/CDA (Task 2) and CWBI (Task 3a) — have different owners and different repositories, so they contend only for HEC's review time, which we manage by submitting small units; (2) maintenance has a dedicated owner whose time is not borrowed from the build streams, and a senior engineer behind him for major fixes; (3) the Project Manager's hours are a floor, not an average — the meeting cadence, note delivery, and Task 5 approvals are fixed obligations and are scheduled first.

Capacity disclosure. Frasier Digital has proposals under evaluation with other agencies whose periods of performance could overlap this one. We size and price every bid against total committed capacity: the hours above are net of every potential concurrent award, the named delivery pod exists to absorb surge, and the Project Manager's commitment to this contract is protected as a floor. We state this proactively because fixed-price, deliverable-based work only functions when capacity claims are honest.

A.3 Responsiveness, customer service, and timeliness

The PWS sets clocks, and we have built our operating rhythm on them rather than on our own:

Obligation	PWS source	How we meet it
Meeting notes within 1 business day	Task 1a	Notes drafted during the meeting from a fixed template; posted same day
Start a maintenance task within 5 days; first work unit within 2 days of starting	Task 5a	Issue triage twice weekly; classification (≤1 h) is the first work unit
Respond to HEC review comments within 5 days	Task 5a	Reviewer comments are the top of the queue; unaddressed comments block new submissions
Status after 1 h classification / 2 h bug work / 4 h modification; draft PR at 4 h	TE4	Time-boxed by the engineer; the PR template carries the elapsed-hours field
STIG state and scan-result responses "promptly"	TE2; Q&A 10	Security folder in the repository is current before each Dev deployment; CWBI requests answered within 2 business days
Notice to proceed before work over 2 h	Task 5a	We ask in the issue; we do not start without the label

Customer service on a contract like this means not consuming the customer's time. HEC has six hours a week for review; our internal review happens before HEC's, every PR is small enough to review in one sitting, and every submission states what it does, what it touches, and how it was tested in the first three lines of the description.

A.4 Reducing performance risk

Table A-4. Risk register.

Risk	Likelihood / impact	Mitigation	Owner
PR #1461 rework is larger than a rebase (161 commits behind; failing CI)	Med / Med	Rebase in the first two weeks on public repos before CWBI access; scope the rework in the first meeting; feature flag keeps `develop` releasable throughout	Randy

Risk	Likelihood / impact	Mitigation	Owner
CWBI access takes the full month (Q&A 11)	High / Low	Month-one work is all local (Factor 2, Tab C); Platform1 ticket templates prepared before award	Scott
Source-of-truth decisions (embargo, offices, constraints) stall in discussion	Med / High	Written comparison with a recommendation at meeting one; ADR with a comment window; the UI increment order puts the dependent work (P1) second, not first	Joseph
Concurrent changes by HEC or other vendors conflict with ours (Q&A 46)	Med / Low	Daily rebase on `develop`; "first merged wins"; design conflicts escalated same day	Randy
HEC review bandwidth limits acceptance rate (Q&A 47)	Med / Med	Small PRs; weekly volume set in the meeting; HEC adding reviewers	Joseph
Oracle RDS behaves differently from the local `oracle-free` image	Low / Med	Every schema PR validated against the integration-test Oracle images and, before Test promotion, in CloudStack or CWBI-Dev; RDS-specific behaviour raised to HEC in the biweekly meeting	Zach
Moonlighting team members' availability shifts	Low / Med	Letters of commitment; hours sized to what each person can sustain; the delivery pod absorbs surge; PM approval required for any substitution, with CO/COR notification	Joseph
Fixed-IP egress for bastion access not ready at award (Q&A 13)	Low / Low	Dedicated static-IP egress provisioned at award; delivering the address to CWBI is a week-1 transition-in item	Joseph

Quality. Definition of done is an accepted work unit. Every code change carries tests in the PWS's preferred locations; OWASP ZAP runs in CI; STIG self-checks precede every Dev deployment; documentation is a work unit, not an afterthought — and the PWS's own list of what counts as documentation (a manual test case anyone can perform, Javadocs, an automated test, formal project documentation) is the checklist in our PR template.

SAM registration and representations. Frasier Digital, LLC is registered and active in SAM.gov (UEI PY8MJ4JPHJ45, CAGE 213L8; registration expires 27 May 2027), and its Representations and Certifications in SAM are current (RFO 4.203-1). No subcontractors or teaming partners are proposed; all personnel named in this proposal perform as members of the Frasier Digital team.

TAB B — Key Personnel Resumes and Experience

B.1 How we read the five roles after the Q&A

The solicitation names five key personnel roles: Project Manager, Senior Oracle DBA, System Engineer/Architect, Senior Forms Developer, and Senior APEX or Java Developer. The Government's answers to questions 7 and 8 settle how two of them apply here: Oracle APEX and Oracle Forms are not used and not planned; the existing components are React, NodeJS/TypeScript, and a Java REST API, "to be expanded, not replaced"; and equivalent experience is acceptable for the Forms Developer and Oracle DBA positions when the person can show "sufficient experience in the direct use of the technologies already in use." We have staffed to that test, using the Government's confirmation that a single individual may be proposed for more than one key personnel role (Q&A 9): the five roles are covered by four named individuals. The Senior Forms Developer and Senior Oracle DBA roles are both filled by Zachary Antosko — his recent work is exactly the kind of data-management web interface TE1 describes, and his database-engineering record (a migration framework with an automated validation engine across 1.15 million production records, delivered with zero downtime) carries the schema side of this design, whose changes are deliberately small, reviewable increments under HEC's review ladder. The Java option of the fifth role is taken. Each resume in the annex opens with the technologies in use on this contract and where the person has used them.

B.2 Role-to-requirement matrix

Table B-1. Key personnel against the technologies in use (Q&A 7–8) and the PWS knowledge areas (instr. 3.2.1.1.1).

Role	Person	Direct use of the technologies in this system	Years	Relevant depth for this contract
Project Manager	Joseph Frasier	React/TypeScript and Node (WeWork, Ecotone, Region 4); authentication and role-based-access backends (Region 4); REST API design; CI/CD; federal and state delivery (USDA-facing app at Deloitte; Louisiana Dept. of Education)	11+	Program manager and hands-on developer on a ground-up platform for thousands of users with role-based access (Region 4, 2024–25); current enterprise AI implementations end-to-end (PwC); single accountable point for a small named team
Senior Oracle DBA (dual role — Q&A 9)	Zachary Antosko	Relational schema design, migration, and validation engineering at production scale: automated ETL with a validation engine across 1.15 M records, zero-downtime cutover; MySQL and PostgreSQL with TypeORM; AWS RDS and DynamoDB; SQL from TypeScript and Node services	5	Owns the schema increments this design needs (embargo table, mask-view extension, test-image currency) — bounded PL/SQL changes submitted small and reviewed by HEC per the PWS ladder — plus the data-integrity discipline the maintenance task depends on
System Engineer / Architect	Scott Carpenter	AWS services and infrastructure-as-code (Amazon SDE; AWS apprenticeship — automated IaC cutting deployment time 96 %); Java and Python; distributed backend systems; Docker; secure-coding and vulnerability documentation for new endpoints	5	Owns the CWBI-side work: sidecar containers, CDK change requests through Platform1, hardened images, health checks, STIG self-assessment. U.S. Army veteran (2014–18)

Role	Person	Direct use of the technologies in this system	Years	Relevant depth for this contract
Senior Forms Developer (React/TypeScript web forms)	Zachary Antosko	React/Redux and TypeScript; Node/NestJS REST backends; PostgreSQL/MySQL with TypeORM; AWS (RDS, DynamoDB, Lambda, S3); Docker; RabbitMQ streaming	5	Built a database-migration framework with automated ETL and a validation engine across 1.15 M records (ID Plans); led a four-engineer platform redesign to pilot in three months (Honeywell); data-management UIs for property, lease, and CRM systems — the shape of the TE1 administrator interface
Senior APEX or Java Developer (Java)	Randy Chong	Java / Spring Boot REST services; AWS ECS, EKS, RDS, CloudWatch in production, including multi-region failover exercises; JUnit; Angular with React exposure; prior QA engineering	4+	Software Engineer II on JPMorgan Chase's digital forms platform (100+ forms): sole developer on a production multi-form completion feature; self-service form-onboarding system; production support and deployment of infrastructure and code. AWS Certified Cloud Practitioner. Owns CDA: the PR #1461 rebase, new `auth/` endpoints, integration tests

All four are U.S. citizens. Public Trust is the clearance level the Government stated (Q&A 11); no team member has a known impediment.

B.3 Why this person for this role

Joseph Frasier, Project Manager. The management risk on a small-business award is the distance between the customer and the decision. Here it is zero: the PM is the owner, ran the last ground-up platform the company delivered as both PM and developer, and will chair every biweekly meeting personally. His hands-on React/TypeScript and authentication-backend background means he reads every pull request, not just the status.

Zachary Antosko, Senior Oracle DBA (dual role). The database work in this design is deliberately bounded: reinstate one table, extend two views, keep the integration-test database images current, and make sure CDA-level filtering never contradicts a VPD policy — small PL/SQL increments, each submitted at the bottom rung of HEC's review ladder and validated by the integration suite against the `database-ready` Oracle image. What that work needs day to day is exactly what Zach's record shows: schema migration executed with automated validation across 1.15 million production records with zero downtime, and the habit of proving data integrity with tooling rather than assertion. Design-bearing database questions go to the biweekly meeting as ADRs before implementation (PWS Coordination), so no schema decision rests on one person's judgment alone.

Scott Carpenter, System Engineer/Architect. The CWBI work is infrastructure-as-code inside someone else's pipeline, with hardened images and a security posture to document. Scott's AWS background is operational (deployments, IaC automation, backend services on distributed systems) and his Army service is a straightforward fit with a Corps of Engineers customer's expectations of discipline in a change process.

Zachary Antosko, Senior Forms Developer. TE1 is a data-administration interface: users, roles, policies, masks, bulk operations, validation. Zach's last two roles were building and migrating data-management systems with React front ends and TypeScript backends, including the validation engineering that keeps bulk operations safe.

Randy Chong, Senior Java Developer. CDA is a Java REST API, and the first job is landing an open pull request that is 161 commits behind on a codebase with a four-schema CI matrix. Randy works in a large-

bank Java/AWS production environment where that kind of disciplined rebase-test-deploy loop is daily practice, and his forms-platform work gives him the domain sense for the authorization endpoints the UI will call.

B.4 Substitution

Key personnel are committed by the letters in the annex. Any substitution during performance requires the Project Manager's approval and written notice to the Contracting Officer and COR with the proposed replacement's resume, and the replacement will meet or exceed the qualifications shown here.

TAB C — Transition Plan

C.1 Principle: nothing of value ever lives only with us

The simplest transition plan is the one that never has to be executed as an event. From the first day of performance, every artifact this contract produces lives in HEC's own repositories, issue trackers, and Confluence: code in USACE's GitHub Enterprise (TE2 Code), decisions as ADRs in `docs/source/decisions`, work status as GitHub issues with the PWS log fields, meeting notes in Confluence, security posture in the repository's security folder, and the load-test harness beside the code it tests. There is no Frasier-Digital-side wiki, backlog, or environment that HEC would need to recover. Transition-out is therefore a matter of closing the loop on work in flight, not of handing over a body of knowledge.

C.2 Transition-in (award to day 30)

Two tracks run in parallel so that access paperwork never idles the engineering work (Q&A 11, 13, 18, 19).

Table C-1. Transition-in schedule.

When	Access track (Government-dependent)	Engineering track (independent of access)
Award week	DD 7798 and DD 2875 submitted for all named staff; Public Trust initiated; Platform1 access request filed; Frasier Digital's fixed egress IP delivered to CWBI for bastion access; read access to the dev CDK requested	Local stacks running on every engineer's machine from the public repositories and docker-compose (Q&A 18); PR #1461 rebase started; `auth-contract` issue triage drafted
Kickoff meeting (week 1–2)	GitHub Enterprise repository access granted (1–2 days after kickoff)	Kickoff is acceptance to start the technical approach (Q&A 48); first ADR drafts (embargo home, office data source, cache topology) presented; first meeting notes posted within 1 business day
Weeks 2–4	Platform1 access (≈1 week+); bastion access when CWBI completes onboarding (≤1 month)	PR #1461 rebase submitted in review-ladder-sized pieces; UI increment U1 in progress; minimal PMP with the standard-app diagram delivered (Q&A 48, 50); first Platform1 ticket drafted and reviewed with HEC before submission
Day 30	All paperwork complete; CWBI-Dev reachable	First accepted work units; Task 5a ready to accept issues; Task 3a change request submitted through Platform1

Onboarding does not gate delivery: the Government confirmed that a contractor "should be able to get started on additional work and planning within existing tools" before console or bastion access lands (Q&A 19), and the first month's engineering is scoped to exactly that.

C.3 Continuity during performance

Three standing practices make any point in the contract a clean handoff point:

- **Every open item is an issue with a status.** Work in progress is a GitHub issue that says what is done, what remains, which branch holds it, and what the next step is. The PWS log fields (requestor, date, notes, status, fix description with tests, hours) are filled as work proceeds, not at invoice time.
- **Every design-bearing choice is an ADR.** Someone who did not attend the meeting can read why the embargo table exists, why the sidecars are in the CDA task, and why the cache has one TTL policy, and can reverse the decision with full context.
- **Every increment is behind a flag until HEC turns it on.** `develop` is releasable at every commit; there is never a half-integrated state that only the contractor knows how to finish.

C.4 Transition-out (final 30 days and close-out)

Table C-2. Transition-out schedule.

When	Activity	Output
T−30 days	Inventory of uncompleted work: every open issue and branch reviewed with HEC in the biweekly meeting; each marked *finish*, *hand off*, or *close*	Handoff list in Confluence with owner and state for each item
T−30 to T−14	*Finish* items completed as work units; *hand off* items brought to a documented stopping point: passing tests, a draft PR with a written "to complete this" section, and an ADR if design-bearing	PRs in review or merged; issues updated
T−14	Documentation audit: administrator guide and training materials (Task 2), load-test README and baseline report (Task 4), security folder and STIG checklist (Task 3), PMP diagram current	Documentation work units accepted
T−14 to T−7	Two handoff sessions inside the biweekly-meeting cadence — one for the authorization system (proxy, OPA, cache, CDA integration, UI), one for the harness and maintenance backlog — recorded as meeting notes with the questions asked and answered	Notes in Confluence; follow-up issues filed
T−7	Maintenance log reconciled: every invoice-referenced issue resolved or annotated with its state; final Task 5 invoice log delivered	Complete log per PWS Task 5a
T−0	Credentials and access returned per CWBI process; final meeting notes; Frasier Digital retains nothing the Government needs	Close-out confirmed in writing to the COR

Option tasks. If Task 3b, 4, or 5b is exercised, the same practices continue and the transition-out schedule shifts to the end of the last exercised task's period of performance (Q&A 32). If an option is not exercised, the base-period close-out above already leaves the Dev deployment, the harness, and the maintenance backlog in a state HEC can carry forward with its own staff or a successor.

Unforeseen early termination. Because the practices in C.3 hold throughout, a transition at any point follows Table C-2 compressed to the notice period; nothing in it depends on a minimum lead time beyond the handoff sessions, which can be held within a week.